EncapsArduino v.2.1

Logiciel d'encapsulage des programmes développés sous IDE Arduino

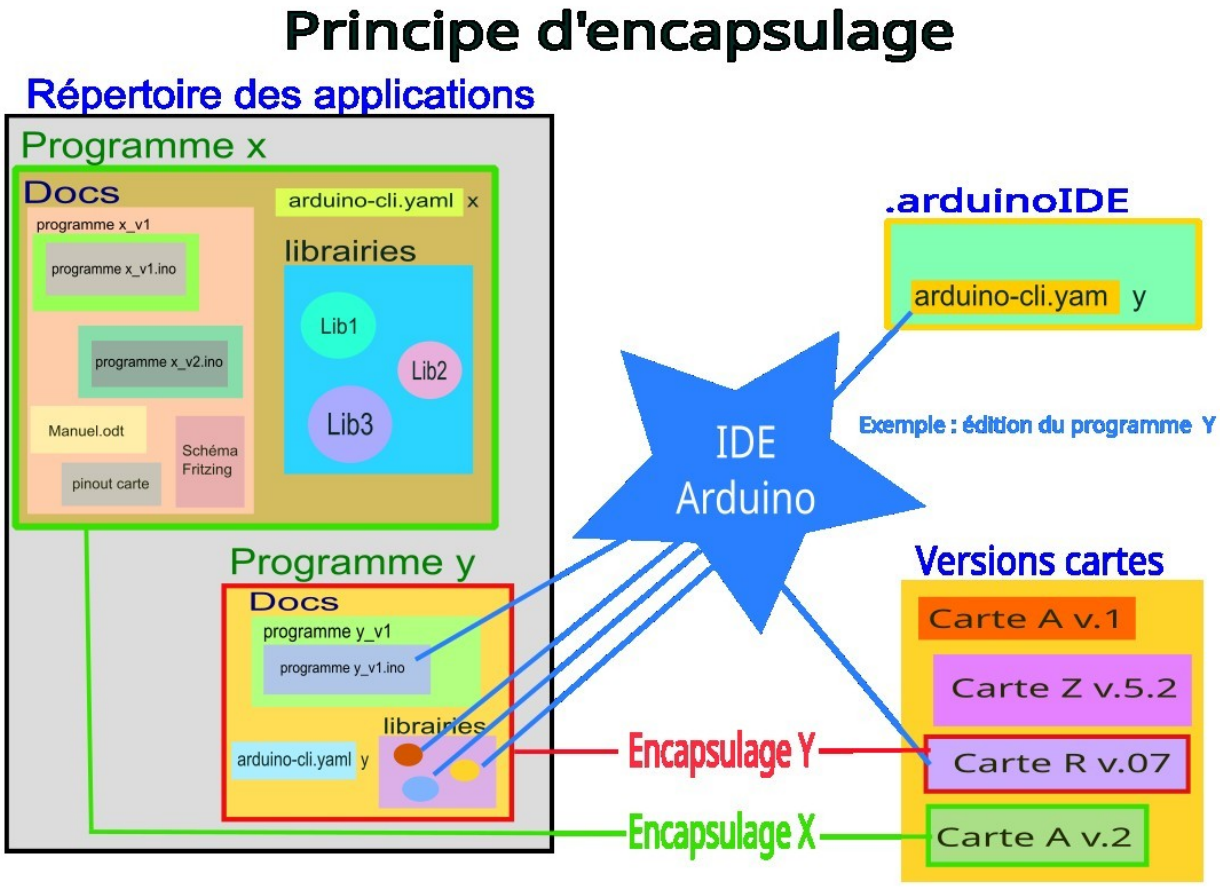
Version multi-langues

Cette application est compatible Linux et Windows.

Manuel de référence

Yves Le Chevalier & Christian Fromentin
pour My Human Kit le 17/03/2026

Ce programme constitue une « fenêtre d'accès » aux programmes Arduino que ce soit lors de la création d'un nouveau programme ou lors de la modification de ceux-ci.



L'encapsulage consiste à isoler la version de carte et les librairies utilisées pour un programme donné, lors du développement d'un programme Arduino afin de retrouver ce même environnement lors de modifications ultérieures sans subir les aléas des évolutions de cartes ou de librairies.

Toutes les cartes dans leurs différentes versions, utilisées par des programmes, sont stockées dans un dossier commun : « Version_cartes ».
Les librairies utilisées par le programme sont stockées dans le dossier librairies du programme.

Le fichier arduino-cli.yaml définit l'environnement pour l'IDE Arduino et il est propre à chaque programme et donc sauvegardé dans le dossier du programme.

Il peut y avoir plusieurs versions du même programme dans le dossier du programme.

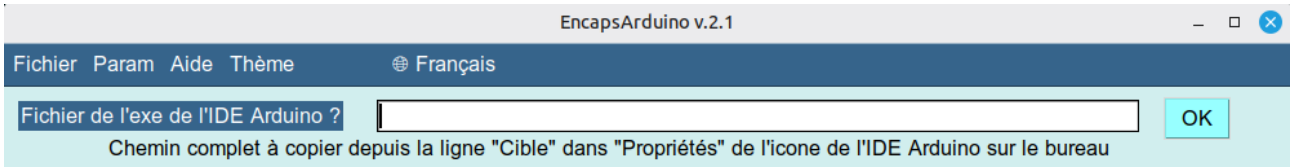
Ce logiciel fonctionne sous Windows et sous Linux en téléchargeant l'exécutable correspondant au système d'exploitation. EncapsArduino2_1[.exe] est accompagné d'une icône qui permettra d'identifier dans l'explorateur de fichiers, les programmes encapsulés. *(CF notes d'installation)*

Par défaut l'interface s'affiche dans la langue de l'utilisateur, mais il est possible de la basculer dans la langue de son choix (Français, Anglais, Italien, Espagnol, Allemand,...etc..)l

1 Questions initiales

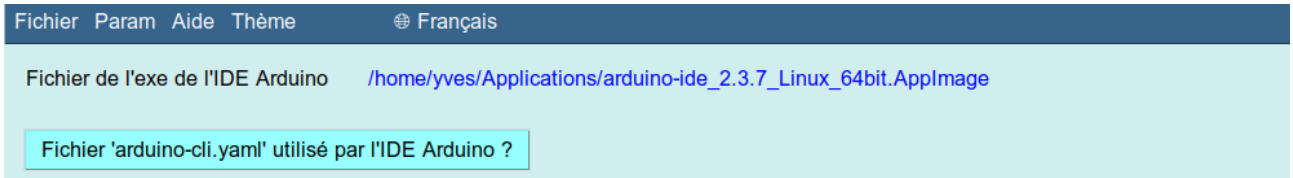
À la première utilisation du logiciel, trois questions sont successivement posées. Elle ne le seront plus par la suite.

Le première question concerne le fichier d'exécution de l'IDE Arduino (Arduino IDE.exe).
On renseigne le champ de saisie par un copier/coller puis on clique sur [OK] ou on fait [Entrée] pour le valider.



Sous Windows, c'est le chemin complet, nom d'exe compris, qui est copié depuis la ligne « Cible » ou « Cible du lien » des propriétés de l'icône de lancement de l'IDE Arduino sur le bureau ou dans le Menu. (Clic droit/Propriétés/Ctrl+C puis Ctrl+V).
Sous Linux on concatène le chemin + / au nom du programme copié dans les Propriétés de l'IDE.

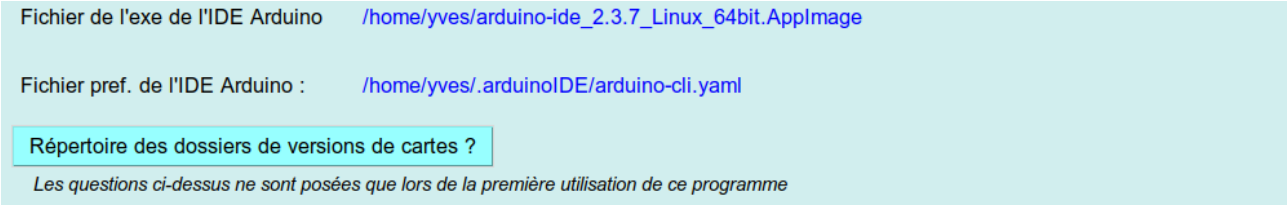
La seconde question posée concerne le fichier utilisé par l'IDE Arduino pour y stocker ses préférences et les chemins d'accès à la carte utilisée et aux librairies appelées par le programme.
C'est le fichier « arduino-cli.yaml » qui se trouve généralement dans le répertoire ...../Utilisateurs/Nom d'utilisateur/.arduinoIDE. En cliquant sur le bouton, l'explorateur de fichiers est automatiquement ouvert pour permettre de trouver ce fichier.
Attention : Ce fichier « arduino-cli.yaml », est un fichier caché, il faut donc dans vos préférences de dossiers, sous Windows cocher la case « Éléments masqués » ou sous Linux,valider« Afficher les fichiers cachés »)



Sous Windows, comme sous Linux, une fois trouvé le fichier « arduino-cli.yaml », on double-clique dessus et le chemin complet avec le nom de fichier apparaît sur la ligne.

La troisième question posée concerne le répertoire où seront stockées les versions de cartes utilisées par les programmes.
Ce répertoire doit avoir été créé auparavant, et peut se situer n'importe où. C'est dans ce répertoire que sera créé automatiquement un dossier pour chaque version de carte utilisée. Il s'agit donc d'un chemin et non pas d'un nom de fichier.

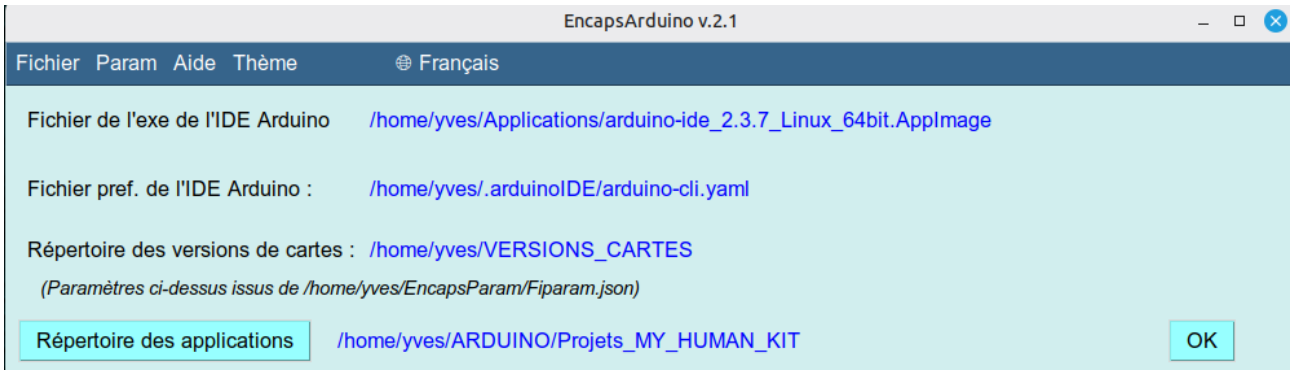
En cliquant sur le bouton, l'explorateur de fichier s'ouvre automatiquement pour permettre de trouver le dossier et sous Windows, comme sous Linux, une fois trouvé le dossier que vous avez créé pour vos cartes, on double-clique dessus et le chemin apparaît sur la ligne.



Une fois que vous avez répondu à ces 3 questions, celles-ci ne vous seront jamais redemandées sauf si vous effacez pour une raison ou une autre le fichier des paramètres (commande située dans la ligne menu de la fenêtre, sous « Param »). En-effet, un fichier Fiparam.json de mémorisation de ces informations a été créé automatiquement à côté de cet exécutable dans le dossier EncapsParam qui contient aussi les dictionnaires des différents langages.

2 Chemin du répertoire des applications Arduino

Une question est ensuite posée pour demander le chemin du répertoire dans lequel sont stockés les dossiers des programmes Arduino.



Ce chemin est mémorisé et sera par la suite automatiquement proposé par défaut. Il suffit de cliquer sur [OK] pour valider la proposition de chemin si celui-ci est le bon.

Cependant vous pouvez avoir plusieurs dossiers de développement selon la destination, le contexte ou le domaine auxquels ils se rattachent.
Il est donc possible de modifier ce répertoire en cliquant sur le bouton [Répertoire des applications]. Ceci ouvre alors l'explorateur de fichiers.

Sous Windows, comme sous Linux, une fois trouvé le dossier des applications, on double-clique dessus et le chemin apparaît sur la ligne.
Il suffit ensuite de cliquer sur [OK] pour valider la désignation du nouveau chemin.

3 Nouveau programme ?

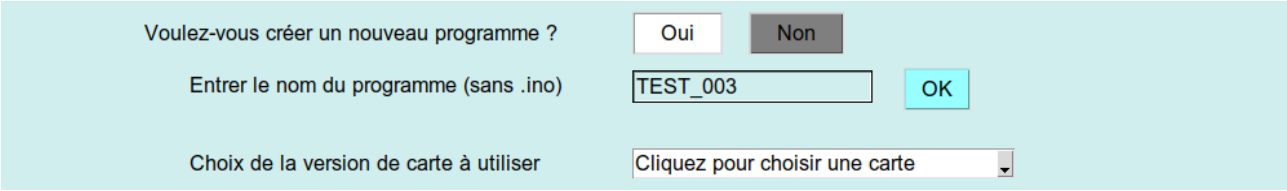
Une fois le répertoire des applications validé, une question s'affiche qui demande si on veut créer un nouveau programme.
On clique soit sur la case Oui soit sur la case Non

Dans ce dernier cas c'est donc une modification de programme ou une nouvelle version d'un programme existant que l'on veut éditer. (voir plus loin « modification de programme »).

3.1 Création d'un nouveau programme

Si on a répondu Oui, il est alors demandé de saisir le nom du programme.
Ce nom de programme est le nom générique, sans numéro de version, ni suffixe « .ino »

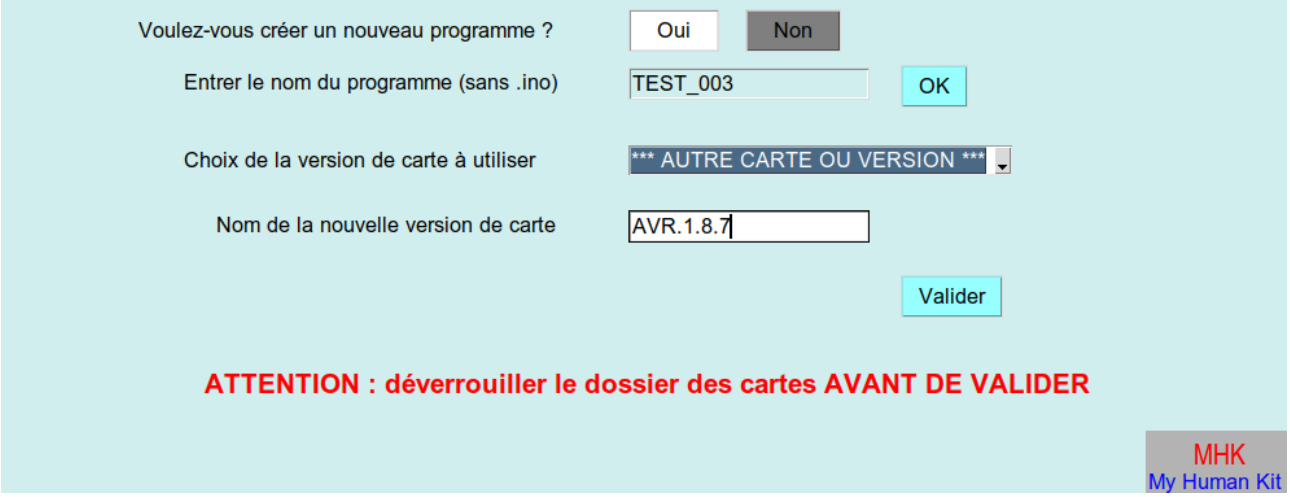
Une fois saisi, le nom du programme doit être validé en cliquant sur [OK] .
Un nom de programme doit faire plus de 3 caractères pour être accepté.
Si ce nom existe déjà dans le répertoire des applications, un message d'erreur est alors affiché.
Dans ce cas,
- soit on s'est trompé et on voulait modifier ce programme ; il suffit alors de cliquer sur Non
- soit ce nom est déjà pris par un autre programme et il faut choisir un autre nom.



Lorsque le nom du nouveau programme est entré, on demande de désigner la version de carte qui sera utilisée par ce programme. Il suffit alors de cliquer dans la fenêtre de la liste déroulante pour voir s'afficher toutes les versions de cartes déjà connues.
La sélection se fait en cliquant sur la version de carte voulue. Il faut ensuite cliquer sur [Valider] pour entériner son choix.

S'il s'agit d'une carte ou d'une version de cartes jamais utilisée, celle-ci ne figure pas dans la liste proposée et il faut cliquer sur « AUTRE CARTE ». Un nouveau champ de saisie apparaît alors pour demander d'entrer le nom et la version de la carte voulue.

Il vous est alors demandé de saisir le nom et le n° de version de la nouvelle version de carte dans un nouveau champ de saisie.



Il est important de saisir dans le nom, le numéro de version de cette carte pour la distinguer par la suite des autres versions de cette même carte. (Cf : exemple de l'illustration ci-dessus)

Attention, si le verrouillage des cartes et librairies Arduino est activé, il faut d'abord déverrouiller ceux-ci avant de Valider sinon le dossier pour la nouvelle version de carte ne pourra pas être créé.
Un message d'alerte au bas de la fenêtre vous prévient dans ce cas.

Le bouton [Valider] valide soit la désignation d'une carte préexistante dans la liste déroulante soit la saisie d'une nouvelle carte dans le second champ.
Des contrôle sont alors effectués pour s'assurer par exemple que la nouvelle carte n'existe pas dans celles qui sont déjà connues ; auquel cas un message d'anomalie serait affiché.

A ce niveau, tous les éléments nécessaires ont été recueillis et le logiciel crée alors un nouveau dossier du nom du programme saisi, dans le répertoire des applications. Ce dossier est caractérisé par une icône «Application Encapsulée ».

Dans ce dossier il stocke une copie du fichier de préférence de l'IDE Arduino (arduino-cli.yaml) configurée pour ce programme et qu'il ne faudra en aucun cas modifier ou effacer.

Il crée ensuite dans le dossier « *nom du programme* » un sous-dossier « Docs » qui contiendra les différentes versions du programme au cours des évolutions de celui-ci mais dans lequel on mettra aussi toute la documentation voulue pour ce programme (Schémas électroniques, photos, mode d'emploi, docs techniques, ...etc).

Dans ce dossier « Docs » un sous-dossier est aussi créé avec le même nom de programme mais suffixé par « _V1 » ie : « *nom du programme_V1* » puisqu'il s'agit de la première version de ce programme que l'on veut écrire.

Dans ce sous dossier « *NomdeProgramme_V1* », un fichier du nom du programme suffixé par « .ino » est créé. le : « *NomdeProgramme_V1.ino* », C'est le squelette du nouveau programme dans lequel est déjà écrit en commentaires la date et la version de carte utilisée.

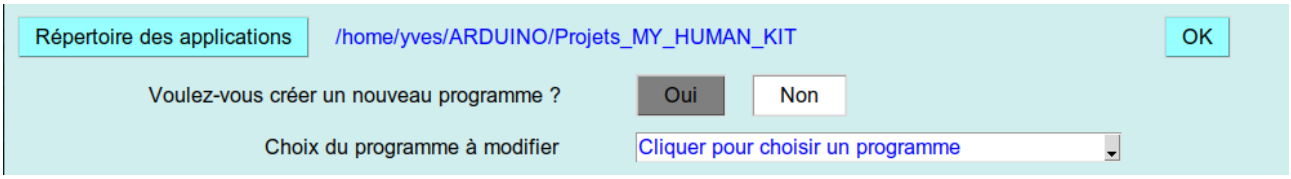
A ce niveau, la fenêtre du logiciel disparaît et ouvre le squelette du programme « *Nomdeprogramme_v1.ino* » dans l'IDE Arduino.

Si une nouvelle carte a été choisie, une fenêtre d'alerte s'ouvre et affiche le message rappelant de faire l'installation de la nouvelle carte avant même de commencer à éditer le code.

3.1 Modification ou nouvelle version d'un programme

NB : La nouvelle version d'un programme sera stockée dans le même dossier que le programme initial (v1) et donc la démarche est ici identique dans les deux cas.

A la réponse Non faite en réponse à la demande « Voulez-vous créer un nouveau programme », une liste déroulante des programmes contenus dans le répertoire des applications sélectionné s'affiche en cliquant dans cette fenêtre.
A noter que seuls les programmes encapsulés sont listés ici.



Il suffit de cliquer sur le nom du programme correspondant et automatiquement l'IDE Arduino s'ouvre sur le sous dossier « Docs » de ce programme tandis que la fenêtre Encapsaduino se ferme.

4 Exemple d'arborescence du répertoire des applications :

Répertoire des applications	*Répertoire général*
ProgrammeX	*Dossier programme X*
Docs	*Dossier de développement*
ProgrammeX_v1	*Dossier du prog.arduino X_v1*
ProgrammeX_v1.ino	*Programme arduino X v1*
arduino-cli.Yaml	Paramètres *IDE arduino modifiés*
librairies	*Librairies spécifiques ajoutées*
Programme Y	*Dossier programme Y*
Docs	*Dossier de développement*
ProgrammeY_v1	*Dossier du progr.arduino Y_v1*
ProgrammeY_v1.ino	*Programme arduino Y_v1*
ProgrammeY_v2	
ProgrammeY_v2.ino	*v2 du prog. ci-dessus*
ManuelY	*Mode d'emploi du prog*
SchémaY1	*Schéma Kikad de la v1*
SchémaY2	*Schéma Kikad de la v2*
arduino-cli.Yaml	*Paramètres IDE arduino modifiés*
librairies	*Librairies spécifiques ajoutées*

5 Remarques diverses

5.1 Lorsqu'on veut faire une nouvelle version d'un programme Prog_v1, il suffit, lorsqu'on arrive dans l'explorateur de fichiers, de faire une copie du programme concerné dans le <u>même</u> répertoire puis de renommer celui-ci en Prog_v2 en incrémentant son numéro de version.

5.2 Lorsqu'on télécharge la version compilée d'EncapArduino , le dossier « Dist » contient l'exécutable du programme. Il est accompagné dans la version Windows d'une icône .ico et dans la version Linux d'une icône .png
Vous pouvez stocker ce dossier où vous voulez dans votre ordinateur.
La première chose à faire ensuite est ce créer un raccourci sur le bureau pour Windows ou un lanceur dans le Menu pour Linux vers cet exécutable. (CF § 6,4)

A l'issue de la première utilisation un fichier « Fiparam.json » sera créé dans ce même répertoire « Dist ». Il ne faut pas le supprimer car il mémorise les chemins des dossiers et fichiers nécessaires à cette application.

5.3 A la création d'un nouveau programme Arduino, le dossier « librairies » figurant dans l'arborescence décrite ci-dessus n'existe pas. Il est automatiquement créé lorsqu'on ajoute des bibliothèques dans l'IDE Arduino (CTRL+Shift+I) et les bibliothèques installées sont automatiquement stockées dans ce dossier « librairies ». Elles resteront donc spécifiques du programme dans leur version d'installation.

5.4 Lorsque l'on clique sur le logo MHK dans le coin inférieur droit de la fenêtre, le navigateur par défaut ouvre le site internet de My Human Kit.

5.5 Lorsque l'on a commencé à développer des programmes Arduino avec ce logiciel d'encapsulage, il est primordial de toujours l'utiliser par la suite, que ce soit pour créer un nouveau programme Arduino ou pour en modifier un autre.

6 Notes d'installation

Il est possible, sous Windows comme sous Linux, d'installer le programme directement en téléchargeant l'exécutable. Mais il est possible aussi, si on dispose d'un environnement Python, de télécharger le source du programme et de le compiler. Bien entendu la procédure n'est pas la même sous Windows et sous Linux.

6.1 Compilation sous Windows

Nota : ceci a été testé avec Windows 10 et Windows 11.

6.1.1 Créer un dossier pour cette application.

6.1.2 Télécharger le source et le copier dans un répertoire réservé à l'application.

6.1.3 Copier aussi l'icône qui accompagne le programme (Encapsule.ico) dans ce même dossier.

6.1.4 Ouvrir une fenêtre Terminal.

6.1.5 Se placer dans le dossier de l'application :
p.ex. : cd F:\Documents\Python\EncapsArduino

6.1.6 Installer PyInstaller si cela n'a pas été déjà fait : pip3 install pyinstaller

6.1.7 Puis lancer la compilation : python -m PyInstaller --clean --onefile --noconsole --collect-all customtkinter --collect-all CTkMessagebox --add-data "F:\Documents\Python\EncapsArduino\Avec_CustomTkinter\Encapsule.ico;." encapsarduino2_1.py

6.2 Installation sous Linux *(Testé avec Ubuntu, Debian et Linux Mint)*

NB : Le chemin vers le dossier Encapsarduino utilisé dans les exemples ci-dessous peut être modifié selon la configuration de vos dossiers.

La première chose à faire est de copier l'icône (Encapsule.png) fournie avec les programme source dans le dossier par défaut des icônes ~/.local/share/icons quitte à devoir créer ce dossier s'il n'existe pas.

6.21 Pour éviter des conflits entre ancienne et nouvelle version, et bien que ce ne soit pas indispensable, il est conseillé d'exécuter un script de nettoyage (clean_build) avant d'effectuer une nouvelle compilation.

On crée ce script bash (si cela n'a pas été déjà fait) en faisant dans une fenêtre Terminal : nano clean_build.sh *Si le script a déjà été fait on passe à 6.24*

6.22 Ce script contient les commandes suivantes :

```
#!/bin/bash
echo "Nettoyage du projet Encapsarduino…"
# Dossier du projet
PROJECT_DIR="$HOME/Documents/Appli_Linux/Encapsarduino"
cd "$PROJECT_DIR" || { echo "Impossible d'accéder au dossier
$PROJECT_DIR"; exit 1; }
BIN_NAME="encapsarduino2_1"   # Nom du programme
echo "Fermeture des instances en cours..."
pkill -f "$BIN_NAME" 2>/dev/null || true
sleep 2
echo "Suppression des dossiers dist/ et build/ et du fichier .spec…"
rm -rf dist build "${BIN_NAME}.spec"
echo "Suppression des fichiers .fuse_hidden*..."
find dist -name ".fuse_hidden*" -type f -exec rm -f {} \; 2>/dev/null ||
true
echo "Nettoyage terminé"
```

6.23 On sauvegarde le script (Ctrl O, Entrée, Ctrl X) et on le rend exécutable en faisant

chmod +x clean_build.sh

6.24 On exécute le script en faisant : ./clean_build.sh

6.3 Compilation elle même : on ouvre une fenêtre Terminal si pas déjà ouverte.

6.31 On se met dans le dossier du programme :

cd ~/Documents/Appli_Linux/Encapsarduino/

6.32 On crée un environnement virtuel : python3 -m venv venv

6.33 On active celui-ci : source venv/bin/activate

6.34 On installe toutes les dépendances Python :

```
pip install pyinstaller
pip install customtkinter
pip install pillow
pip install pyyaml
```

6.35 On lance la compilation avec PyInstaller :

```
python -m PyInstaller encapsarduino2_1.py \
  --onefile \
  --noconsole \
  --add-data "Encapsule.png:." \
  --hidden-import customtkinter \
  --hidden-import PIL._tkinter_finder \
  --hidden-import yaml
```

6.36 On donne les droits d'exécution : chmod +x dist/encapsarduino2_1

6.37 L'exécutable est alors disponible dans le dossier dist.
On peut l'exécuter en faisant : ./dist/encapsarduino2_1

6.4 Ajouter l'application au Menu Linux

6.41 Pour cela on va créer un lanceur desktop avec les paramètres de l'application en faisant :

```
nano ~/.local/share/applications/encapsarduino2_1.desktop
```

6.42 On renseigne les paramètres de l'application avec ceci :

```
[Desktop Entry]
Name=EncapsArduino
Exec=/home/Utilisateur/Documents/Appli_Linux/Encapsarduino/dist/
encapsarduino2_1
Icon=/home/Utilisateur/.local/share/icons/Encapsule.png
Terminal=false
Type=Application
Categories=Development;Utility;
```

6.43 On sauvegarde le script (Ctrl O, Entrée, Ctrl X)

6.44 Puis on force sa prise en compte en faisant

```
update-desktop-database ~/.local/share/applications
```

=> *l'application apparaît dans le menu avec son icône spécifique.*

6.45 Attention à bien copier le dossier EncapsParam dans votre « Dossier personnel » (/home/*utilisateur*) lors de la mise en place de ce lanceur.

7 Complément à Encapsarduino : Verrouillage cartes et librairies

L'encapsulage des programmes Arduino avec leurs librairies et les cartes utilisées ne suffit pas à protéger les librairies et les cartes d'une mise à jour évolutive proposée par l'IDE Arduino.
Un premier niveau de protection peut être activé en empêchant les notifications des mises à jour dans l'IDE.
Cette action peut être activée sous Windows comme sous Linux de la même manière.

Tout d'abord il faut accéder aux paramètres de l'IDE Arduino en démarrant celle-ci et en faisant **[Ctrl] [Shift] P**.
Dans la liste qui s'affiche, il faut rechercher **Open Settings** dans la ligne de recherche en haut de la fenêtre puis cliquer sur la ligne **Open Settings (UI)**.
Dans la colonne de gauche, on active l'onglet Utilisateur s'il n'est pas activé.
On clique ensuite sur **Extensions** pour déployer ce sous-menu puis on clique sur **Arduino**.
On déroule ensuite la liste des paramètres dans la colonne de droite jusqu'à trouver **Check For Updates** afin de décocher la case de ce paramètre.

Ceci évite d'avoir des notifications intempestives signalant des mises à jours possibles des librairies et des cartes utilisées par le programme Arduino en cours de développement ou de modification.
Toutefois cela n'empêche pas de pouvoir mettre à jour par erreur les cartes ou les librairies.

7.1 Solution Linux

Une solution existe sous Linux pour empêcher cela en rendant non modifiables les dossiers des cartes et des librairies utilisés (désignés dans le fichier arduino-cli.yaml de l'IDE).
Cette solution a été testée sous Linux Mint 22.2 et 22.3 et Debian.

Elle consiste en un script Bash qui lit le fichier arduino-cli.yaml et qui verrouille (ou déverrouille) le dossier qui contient tous les dossiers des cartes (et pas seulement celle utilisée par le programme Arduino en cours) ainsi que les librairies utilisées par le programme Arduino.
Ce script fonctionne en mode « bascule » et déverrouille ou verrouille alternativement les dossiers selon leur état initial.
Il affiche une notification décrivant l'action qui a été effectuée.

(NB : Ce script pour Linux a été amélioré et au vu de sa complexité a été largement documenté le 02/03/2026 sous le nouveau nom arduino_lock.sh).

```
#!/bin/bash
#
==========================================================================
========
# arduino_lock.sh
# Script de verrouillage/déverrouillage des dossiers Arduino
# Protège les cartes et librairies contre les mises à jour involontaires
#
==========================================================================
========

# ---------------------------------------------------------------------------
# VÉRIFICATION DES DROITS ADMINISTRATEUR
# ---------------------------------------------------------------------------
```

```
# Modifier les permissions de fichiers nécessite des droits root (administrateur).
# "$EUID" contient l'identifiant de l'utilisateur courant (0 = root).
# Si on n'est pas root, on relance ce même script ($0) via pkexec,
# qui affiche une fenêtre graphique de demande de mot de passe.
if [ "$EUID" -ne 0 ]; then
    pkexec "$0"
    exit 0
fi

# ---------------------------------------------------------------------------
# IDENTIFICATION DYNAMIQUE DE L'UTILISATEUR RÉEL
# ---------------------------------------------------------------------------
# Quand pkexec élève les privilèges, il remplit automatiquement la variable
# PKEXEC_UID avec l'UID (numéro) de l'utilisateur qui a lancé le script.
# On s'en sert pour retrouver son nom (REAL_USER) et son dossier home (HOME_DIR),
# sans jamais avoir à écrire un nom d'utilisateur en dur dans le script.
#
# "getent passwd" interroge la base des utilisateurs du système.
# "cut -d: -f1" extrait le 1er champ (le nom) séparé par des ":"
# "cut -d: -f6" extrait le 6ème champ (le dossier home)
REAL_USER=$(getent passwd "$PKEXEC_UID" | cut -d: -f1)
HOME_DIR=$(getent passwd "$PKEXEC_UID"  | cut -d: -f6)

# Sécurité : si on n'a pas pu identifier l'utilisateur, on quitte immédiatement
if [ -z "$REAL_USER" ] || [ -z "$HOME_DIR" ]; then
    echo "Erreur : impossible d'identifier l'utilisateur réel (PKEXEC_UID=$PKEXEC_UID)"
    exit 1
fi

# ---------------------------------------------------------------------------
# FONCTION : ENVOYER UNE NOTIFICATION VISUELLE À L'UTILISATEUR
# ---------------------------------------------------------------------------
# Comme le script tourne en root, il n'a pas accès au bureau de l'utilisateur.
# On doit donc "emprunter" la session graphique de l'utilisateur réel pour
# afficher une notification dans sa barre système.
#
# Paramètres :
#   $1 = titre de la notification
#   $2 = message de la notification
notify_user() {
    # PKEXEC_UID contient déjà l'UID de l'utilisateur réel — on l'utilise
    # directement pour trouver le bus de communication de son bureau.
    # su -c "commande" $REAL_USER → exécute la commande en tant que l'utilisateur réel
    su -c "DISPLAY=:0 DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/$
{PKEXEC_UID}/bus \
          notify-send '$1' '$2'" "$REAL_USER"
}

# ---------------------------------------------------------------------------
# LECTURE DU FICHIER DE CONFIGURATION arduino-cli.yaml
# ---------------------------------------------------------------------------

# Le chemin du fichier yaml est construit à partir du home de l'utilisateur réel,
# détecté dynamiquement — plus aucun nom d'utilisateur codé en dur.
YAML="${HOME_DIR}/.arduinoIDE/arduino-cli.yaml"
```

```
# awk '/data:/ {print $2}' : cherche la ligne contenant "data:" et extrait
# le 2ème mot (le chemin du dossier de la carte)
# tr -d '\r' : supprime les éventuels retours chariot Windows (\r)
RAW_DATA_PATH=$(awk '/data:/ {print $2}' "$YAML" | tr -d '\r')

# Sécurité : si le chemin est vide (ligne "data:" absente ou mal formatée),
# on avertit et on quitte proprement
if [ -z "$RAW_DATA_PATH" ]; then
    notify_user "Erreur" "Chemin YAML introuvable"
    exit 1
fi

# dirname extrait le dossier parent d'un chemin.
# Ex: /home/alice/VERSIONS_CARTES/ESP32.3.1.3 → /home/alice/VERSIONS_CARTES
# On protège ainsi TOUT le dossier des versions de cartes, pas juste une carte.
CARDS_DIR=$(dirname "$RAW_DATA_PATH")

# On extrait le chemin "user:" du yaml (dossier du projet en cours)
USER_DIR=$(awk '/user:/ {print $2}' "$YAML" | tr -d '\r')

# Le dossier des librairies est toujours le sous-dossier "libraries"
# à l'intérieur du dossier projet
LIB_DIR="$USER_DIR/libraries"

# ---------------------------------------------------------------------------
# FONCTION : VERROUILLER (passer en lecture seule)
# ---------------------------------------------------------------------------
lock() {
    # chmod = change mode (modifie les permissions d'un fichier ou dossier)
    # -R    = Récursif : applique le changement à tous les sous-dossiers et fichiers
    # a-w   = "all minus write" : retire le droit d'écriture (w) à TOUS
    #         (a = all : utilisateur + groupe + autres)
    chmod -R a-w "$CARDS_DIR"

    # On ne verrouille les librairies que si le dossier existe
    # (il peut ne pas exister si le projet n'en utilise pas)
    if [ -d "$LIB_DIR" ]; then
        chmod -R a-w "$LIB_DIR"
    fi

    notify_user "Arduino" "🔒 MODE LECTURE SEULE (Verrouillé)"
}

# ---------------------------------------------------------------------------
# FONCTION : DÉVERROUILLER (remettre en mode écriture)
# ---------------------------------------------------------------------------
unlock() {
    # u+w = "user plus write" : redonne le droit d'écriture (w) uniquement
    #       au propriétaire (u = user), pas au groupe ni aux autres.
    #       C'est plus sûr que de tout réouvrir avec "a+w".
    chmod -R u+w "$CARDS_DIR"

    if [ -d "$LIB_DIR" ]; then
        chmod -R u+w "$LIB_DIR"
```

```
    fi

    notify_user "Arduino" "🔓 MODE ÉDITION (Déverrouillé)"
}

# ---------------------------------------------------------------------------
# LOGIQUE PRINCIPALE : BASCULE (TOGGLE)
# ---------------------------------------------------------------------------
# Le script fonctionne comme un interrupteur :
# - Si le dossier est actuellement accessible en écriture → on verrouille
# - Si le dossier est déjà verrouillé → on déverrouille
#
# ATTENTION : on ne peut pas utiliser [ -w "$CARDS_DIR" ] directement ici,
# car le script tourne en root, et root peut TOUJOURS écrire partout,
# même sur un dossier verrouillé en a-w. Le test retournerait donc
# toujours "vrai" et le script verrouillerait à chaque exécution.
#
# Solution : on effectue le test en tant que l'utilisateur RÉEL (REAL_USER)
# via "su -c", qui lui voit les vraies permissions appliquées.
# La commande "test -w" retourne 0 (succès) si accessible en écriture.
if su -c "test -w '$CARDS_DIR'" "$REAL_USER"; then
    lock    # Dossier accessible en écriture pour l'utilisateur → on verrouille
else
    unlock  # Dossier verrouillé pour l'utilisateur → on déverrouille
fi
```

Ce script peut être stocké dans le dossier de votre choix.
p.ex. /home/*Utilisateur*/Scripts/arduino-lock.sh
Il est lancé à la demande en créant un lanceur dans le Menu ou dans le Tableau de Linux Mint. Le lanceur peut être celui-ci :

```
[Desktop Entry]
Name=Arduino Verrouillage
Comment=Verrouille ou déverrouille l'environnement Arduino
Exec=/home/Utilisateur/Scripts/arduino-lock.sh
Icon=preferences-system
Terminal=false
Type=Application
Categories=Electronics;
```

Il est enregistré dans le dossier de votre choix mais préférablement dans
/home/*Utilisateur*/.local/share/applications

NB : Si jamais, pour une raison ou une autre, le script de verrouillage ne pouvait plus être exécuté et qu'on se trouve en mode « verrouillé », il est possible de basculer en mode « déverrouillé » par ces deux commandes :

```
chmod -R u+w ~/votre dossier de répertoire des cartes
chmod -R u+w ~/ARDUINO/votre dossier de programme/librairies
```

72 Solution Windows

La solution sous Windows est analogue à celle pratiquée sous Linux. Elle consiste à rendre non modifiables le dossier des cartes et celui des librairies utilisés (désignés dans le fichier arduino-cli.yaml de l'IDE).
Cette solution a été testée sous Windows 11.

Dans un répertoire de votre choix, créer un fichier texte dans lequel vous copierez le script ci-après puis enregistrez le sous un nom xxxx.ps1 (p.ex. arduino-toggle.ps1).

Pour pouvoir exécuter ce script en mode administrateur ouvrez un terminal administrateur (clic droit dans l'icône Windows « démarrer ») et taper ceci :
Set-ExecutionPolicy RemoteSigned (répondre « oui » à la question posée)

On peut exécuter le script en faisant un clic droit sur celui-ci et en sélectionnant « **Exécuter avec PowerShel**l ».

Il est cependant plus confortable de créer un icône de lancement sur le bureau.
Pour cela : - faire un clic droit sur le bureau
- choisir « Nouveau... Raccourci »
- dans le champ « Entrez l'emplacement de l'élément » copier ceci :

powershell.exe -ExecutionPolicy Bypass -File "C:\xxxxx\arduino-toggle.ps1"
en remplaçant xxxxx par le chemin vers le script tel que vous l'avez enregistré.
- cliquer sur « suivant »
- donner un nom à votre icône.(p.ex. « Verrouillage On-Off IDE Arduino »)
- changer éventuellement l'icône (propriétés/raccourci/changer d'icône)
- Dans propriété du raccourci cliquer sur « Avancé »
- Cocher la case « Exécuter en tant qu'administrateur »

A l'exécution du script, une fenêtre terminal s'affiche pour vous montrer le bon déroulement du processus, ou les erreurs éventuelles. Faites Entrée pour fermer la fenêtre.

Une notification s'affiche alors en bas et à droite de l'écran pour indiquer l'état des dossiers ; soit « verrouillé » soit « déverrouillé ».
Il faut toutefois avoir préalablement activé les notifications (Système > Notifications).

Voici le script à copier dans arduino-toggle.ps1 (si vous avez gardé le nom donné en exemple) :

```
# 1. Privilèges Administrateur (Méthode robuste)
if (-not ([Security.Principal.WindowsPrincipal]
[Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuil
tInRole]::Administrator)) {
   Start-Process powershell.exe -ArgumentList "-NoProfile -ExecutionPolicy Bypass -File
`"$PSCommandPath`"" -Verb RunAs
   exit
}

# 2. Récupérer le VRAI nom d'utilisateur (celui de la session active)
$RealUser = [System.Security.Principal.WindowsIdentity]::GetCurrent().Name.Split('\')[1]
if ($env:USERNAME_SESSION) { $RealUser = $env:USERNAME_SESSION }
elseif ($env:USERPROFILE) { $RealUser = $env:USERPROFILE.Split('\')[-1] }
$YAML = "C:\Users\$RealUser\.arduinoIDE\arduino-cli.yaml"

Write-Host "--- Diagnostic ---" -ForegroundColor Yellow
```

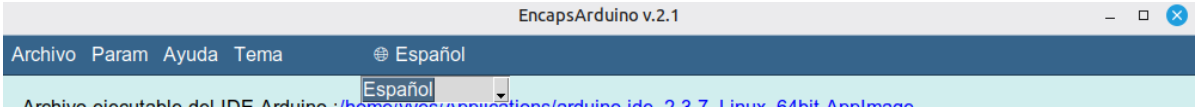
```
Write-Host "Utilisateur detecte : $RealUser"
Write-Host "Fichier YAML cible  : $YAML"

# Préparation de l'objet notification
Add-Type -AssemblyName System.Windows.Forms
$balloon = New-Object System.Windows.Forms.NotifyIcon
$balloon.Icon = [System.Drawing.Icon]::ExtractAssociatedIcon((Get-Process -id $pid).Path)
$balloon.Visible = $true

if (-not (Test-Path $YAML)) {
   Write-Host "ERREUR : Fichier YAML introuvable." -ForegroundColor Red
   $balloon.ShowBalloonTip(5000, "Arduino IDE", "ERREUR : YAML introuvable", "Error")
   Pause ; exit
}

# 3. Extraction des chemins
$DataPath = ""
$UserPath = ""
foreach ($line in Get-Content $YAML) {
   if ($line.Trim().StartsWith("data:")) {
      $DataPath = $line.Replace("data:", "").Trim().Replace("'", '')
   }
   if ($line.Trim().StartsWith("user:")) {
      $UserPath = $line.Replace("user:", "").Trim().Replace("'", '')
   }
}

if (-not $DataPath) {
   Write-Host "ERREUR : 'Dossier des cartes ' non trouve dans le fichier." -
ForegroundColor Red
   Get-Content $YAML | Select-Object -First 10
   Pause ; exit
}

# 4. Définition des dossiers
$CardsDir = Split-Path -Path $DataPath -Parent
$LibDir = Join-Path -Path $UserPath -ChildPath "libraries"

Write-Host "Dossier des cartes : $CardsDir" -ForegroundColor Cyan
Write-Host "Dossier des librairies   : $LibDir" -ForegroundColor Cyan

# 5. Fonctions de Verrouillage
$EveryoneSID = "*S-1-1-0"

function Lock-Folders {
   Write-Host "Passage en Lecture Seule..." -ForegroundColor Yellow

   # 1. On réinitialise les permissions pour qu'elles soient propres (:reset)
   icacls "$CardsDir" /reset /T /C /Q

   # 2. On donne uniquement le droit de Lecture/Exécution (RX) à "Tout le monde"
   # /inheritance:r supprime les droits hérités du dossier parent (souvent trop permissifs)
   # :RX = Read & Execute
   icacls "$CardsDir" /inheritance:r /grant:r "${EveryoneSID}:RX" /T /C /Q
```

```
    if (Test-Path $LibDir) {
        icacls "$LibDir" /reset /T /C /Q
        icacls "$LibDir" /inheritance:r /grant:r "${EveryoneSID}:RX" /T /C /Q
    }

    Write-Host "ETAT : VERROUILLE (Lecture seule stricte)" -ForegroundColor Red
    $balloon.ShowBalloonTip(5000, "Arduino IDE", "ETAT : VERROUILLE (Lecture Seule)",
"Info")
}

function Unlock-Folders {
    Write-Host "Retablissement des droits complets..." -ForegroundColor Yellow

    # On remet l'héritage normal (ce qui redonne les droits d'écriture par défaut de Windows)
    icacls "$CardsDir" /reset /T /C /Q
    icacls "$CardsDir" /inheritance:e /T /C /Q

    if (Test-Path $LibDir) {
        icacls "$LibDir" /reset /T /C /Q
        icacls "$LibDir" /inheritance:e /T /C /Q
    }

    Write-Host "ETAT : DEVERROUILLE (Controle total)" -ForegroundColor Green
    $balloon.ShowBalloonTip(5000, "Arduino IDE", "ETAT : DEVERROUILLE (Edition)",
"Info")
}


# 6. Logique de bascule
$TestFile = Join-Path -Path $CardsDir -ChildPath ".lock_test"
try {
    $null = New-Item -Path $TestFile -ItemType File -ErrorAction Stop
    Remove-Item $TestFile -Force
    Lock-Folders
} catch {
    Unlock-Folders
}

# Petit délai pour laisser la notification apparaître avant la fin du script
Start-Sleep -Seconds 1
 Write-Host "`nTermine. Appuyez sur une touche..."
 $null = $Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown")
```

NB : Il est possible de mettre en commentaires les 2 dernières lignes ci-dessus pour ne pas avoir à fermer la fenêtre à la fin du traitement.

8 Gestion des langues de l'interface

Par défaut la langue utilisée dans cette application sera celle de la localisation de l'utilisateur, mais il est possible à tout moment de changer de langue en cliquant sur la langue affichée dans la barre de menu.



Une fois la langue modifiée, celle-ci sera automatiquement ré-utilisée par défaut au lancement de l'application (sauf si on efface le fichier paramètres).

Lorsque l'on clique sur la langue affichée dans la barre de menu, la liste des langues connues par l'application s'affiche et il suffit ensuite de cliquer sur la nouvelle langue désirée pour que tous les textes soient automatiquement affichés dans cette langue.

Cette fonctionnalité repose sur la présence dans le dossier EncapsParam d'un fichier .json spécifique à chaque langue et qui contient le dictionnaire de la langue.
Il existe en outre dans ce même dossier un fichier «langthem.json » qui contient la liste des langues connues et donc des dictionnaires correspondants.

Celui-ci se présente ainsi :

```
{
  "languages": [
    { "name": "Français", "file": "lang_FR.json", "code": "FR" },
    { "name": "English",  "file": "lang_EN.json", "code": "EN" },
    { "name": "Español",  "file": "lang_ES.json", "code": "ES" },
    { "name": "Italiano", "file": "lang_IT.json", "code": "IT" },
    { "name": "Deutsch",  "file": "lang_DE.json", "code": "DE" }
  ]
}
```

Pour ajouter une nouvelle langue XX, il n'est pas besoin de modifier le code ni de re-compiler l'application. Il suffit, pour cela :

1 - D'ajouter une entrée avec "code" : "XX" pour la nouvelle langue considérée dans le paragraphe « langages » du fichier «langthem.json »
avec le "name": correspondant au nom de la langue (exprimée dans celle-ci)
et avec "file": "lang_XX.json" qui est le nom du dictionnaire.

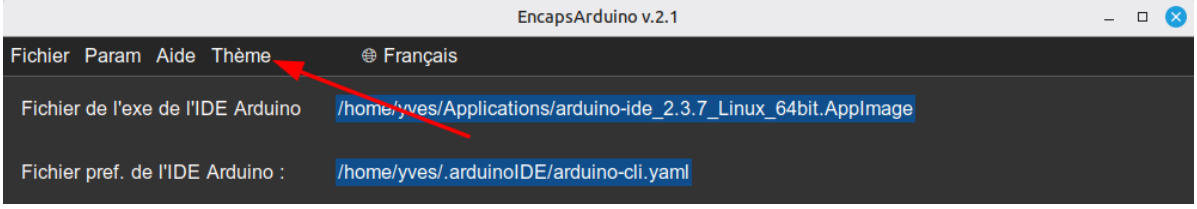
2 - De créer un nouveau fichier-dictionnaire lang_XX.json pour la nouvelle langue.

Ce fichier pourra être généré à partir de l'un des fichiers existants et en traduisant l'ensemble des lignes du dictionnaire de référence à l'exception des clefs.
(Chapt GPT sait très bien traduire ce qu'il faut dans un dictionnaire .json à condition de bien de le lui préciser dans le prompt.)

9 Thèmes

Cette application peut être affichée selon deux modes : clair ou foncé.
Selon le thème choisi par l'utilisateur, toutes les couleurs de l'interface basculent dans le mode correspondant.

Le choix du thème proposé dans la barre de menu est activé par un clic sur l'un des deux boutons correspondants.



Lorsqu'un thème a été choisi, celui-ci sera automatiquement repris au prochain lancement de l'application (sauf si on efface le fichier paramètres).

Les couleurs appliquée à chacun des thèmes sont définies dans un dictionnaire des couleurs stocké dans le paragraphe « themes » du fichier «langthem.json » .

Il est par conséquent possible à l'utilisateur averti de modifier les couleurs proposées par défaut pour chacun des deux thèmes sous réserve de bien respecter la syntaxe json et le nommage des clés. On aura grandement intérêt, dans ce cas, à faire une copie de sauvegarde du fichier avant toute modification.

« Ce n'est pas la peine d'avoir trouvé de si belles chaussures pour continuer à marcher pieds nus »
(proverbe indigène) 🙂